

Schéma d'API

Overdrive

Aventix
Epitech Paris
9 février 2026

Sommaire

OverDrive – Schéma d'API

1- Principes généraux : REST vs WebSocket

- 1- Principes généraux : REST vs WebSocket
- 2- WebSocket : diffusion live
- 3- Schéma : Architecture globale (REST + WebSocket)

2- API REST : endpoints probables

- 1- Authentification
- 2- Providers et catalogue des courses
- 3- Presets et layouts
- 4- Widgets d'un preset
- 5- Highlights (optionnel)
- 6- Schéma : Modèle de données
- 7- Schéma : Séquence d'authentification

3- WebSocket : diffusion temps réel

- 1- Connexion
- 2- Types de messages envoyés
- 3- Gestion de la charge et optimisation
- 4- Schéma : Connexion WebSocket + souscription à une session
- 5- Schéma : Format des messages WebSocket
- 6- Schéma : Optimisation de charge (diffusion différenciée)

4- Principes techniques recommandés

- 1- Versionnement
- 2- Documentation (OpenAPI / Swagger)
- 3- Validation et schémas JSON
- 4- Rate limiting (REST)
- 5- Gestion d'erreurs standardisée
- 6- Contrôle d'accès

5-Conclusion

Objectif

L'API d'OverDrive est conçue pour répondre à deux besoins majeurs et complémentaires :

- **Gérer le produit et les données persistantes** (comptes, presets, catalogue des courses, configuration utilisateur).
- **Diffuser les données live en temps réel** (classement, télémétrie, événements de course) avec une latence minimale.

Pour atteindre ces objectifs, OverDrive adopte une architecture standard et robuste basée sur :

- une **API REST** pour toutes les opérations classiques et persistantes,
- un **canal WebSocket** pour le streaming temps réel.

Cette séparation est essentielle dans une application de sport automobile, car elle évite de surcharger l'API REST avec des requêtes répétées (polling), tout en assurant une expérience fluide sur tous les clients (VR, mobile, web).

1- Principes généraux : REST vs WebSocket

1- API REST : gestion produit et configuration

L'API REST est utilisée pour toutes les opérations qui ne nécessitent pas une mise à jour constante, notamment :

- authentification,
- récupération du profil utilisateur,
- gestion des presets et layouts,
- catalogue des courses et sessions,
- configuration des providers,
- accès aux données persistantes (highlights, replays, etc.).

Le REST est particulièrement adapté car il est :

- simple à versionner,
- facile à documenter,
- compatible avec tous les clients,
- stable pour des opérations CRUD (Create, Read, Update, Delete).

2- WebSocket : diffusion live

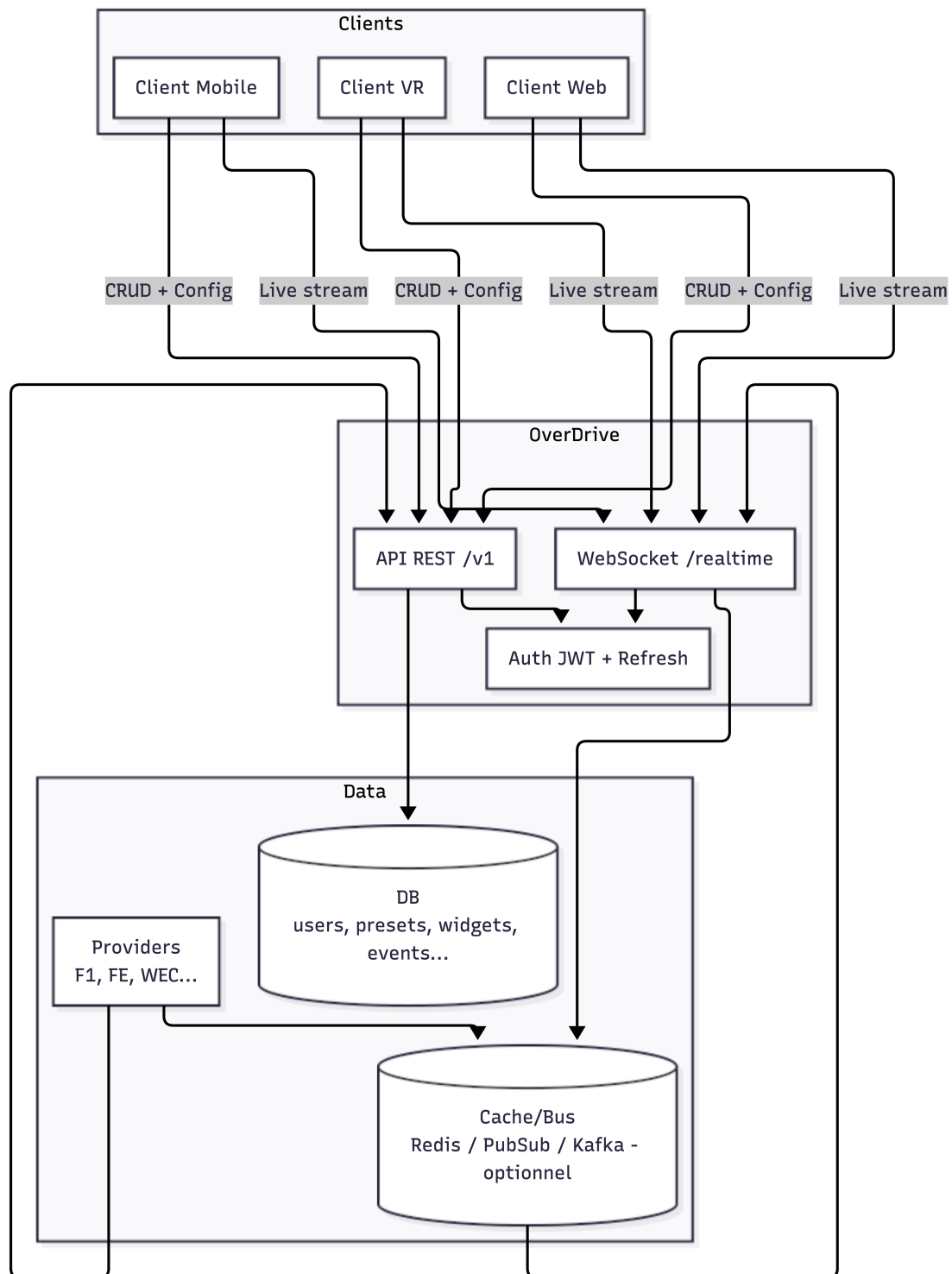
Le WebSocket est utilisé pour la diffusion en temps réel, car il permet :

- une connexion persistante,
- l'envoi automatique des mises à jour sans requête client,
- une latence réduite,
- un fonctionnement optimal pour des données très fréquentes.

Ce canal est utilisé pour diffuser :

- état de session (lap, drapeaux, météo, statut piste),
- mises à jour de classement,
- écarts,
- télémétrie,
- événements importants (highlight_event).

3- Shéma : Architecture globale (REST + WebSocket)



2- API REST : endpoints probables

L'API REST d'OverDrive est organisée autour de grands blocs fonctionnels.

1- Authentification

L'authentification couvre les opérations essentielles de gestion utilisateur :

- création de compte,
- connexion,
- déconnexion,
- récupération du profil.

Endpoints principaux :

- **POST /auth/register**
- **POST /auth/login**
- **POST /auth/logout**
- **GET /me**

L'authentification peut s'appuyer sur :

- un token d'accès (type JWT),
- un mécanisme de refresh token pour maintenir une session utilisateur sans compromettre la sécurité.

Ce système permet un bon équilibre entre :

- sécurité (expiration courte des access tokens),
- confort utilisateur (refresh token).

2- Providers et catalogue des courses

OverDrive étant multi-championnats, le client doit pouvoir accéder à un catalogue structuré.

Endpoints principaux :

- **GET /providers (liste des championnats disponibles)**
- **GET /events?provider=f1&year=2026**
- **GET /events/{eventId}**
- **GET /events/{eventId}/sessions**

Ces routes permettent :

- d'afficher un calendrier annuel,
- de naviguer dans les événements passés,
- de préparer une session à suivre en live,
- de lier les sessions (practice/quali/race) à un événement.

3- Presets et layouts

La personnalisation étant un élément central d'OverDrive, la gestion des presets doit être complète et accessible depuis tous les clients.

Endpoints principaux :

- **GET /presets**
- **POST /presets**
- **GET /presets/{presetId}**
- **PUT /presets/{presetId}**
- **DELETE /presets/{presetId}**

Ces endpoints permettent :

- de créer plusieurs presets,
- de sauvegarder et restaurer des layouts,
- de définir un preset par défaut,
- d'assurer une cohérence entre VR, mobile et web.

4- Widgets d'un preset

Chaque preset est composé d'un ensemble de widgets configurables. Le client doit pouvoir gérer ces widgets de manière structurée.

Endpoints principaux :

- **GET /presets/{presetId}/widgets**
- **POST /presets/{presetId}/widgets**
- **PUT /presets/{presetId}/widgets/{widgetId}**
- **DELETE /presets/{presetId}/widgets/{widgetId}**

Ces endpoints couvrent :

- l'ajout de widgets,
- la suppression,
- la mise à jour des paramètres (settings),
- la sauvegarde des positions et transformations VR.

Cette logique garantit une personnalisation complète sans dépendre d'un client spécifique.

5- Highlights (optionnel)

Si le backend stocke des événements importants en base, un endpoint dédié permet d'y accéder.

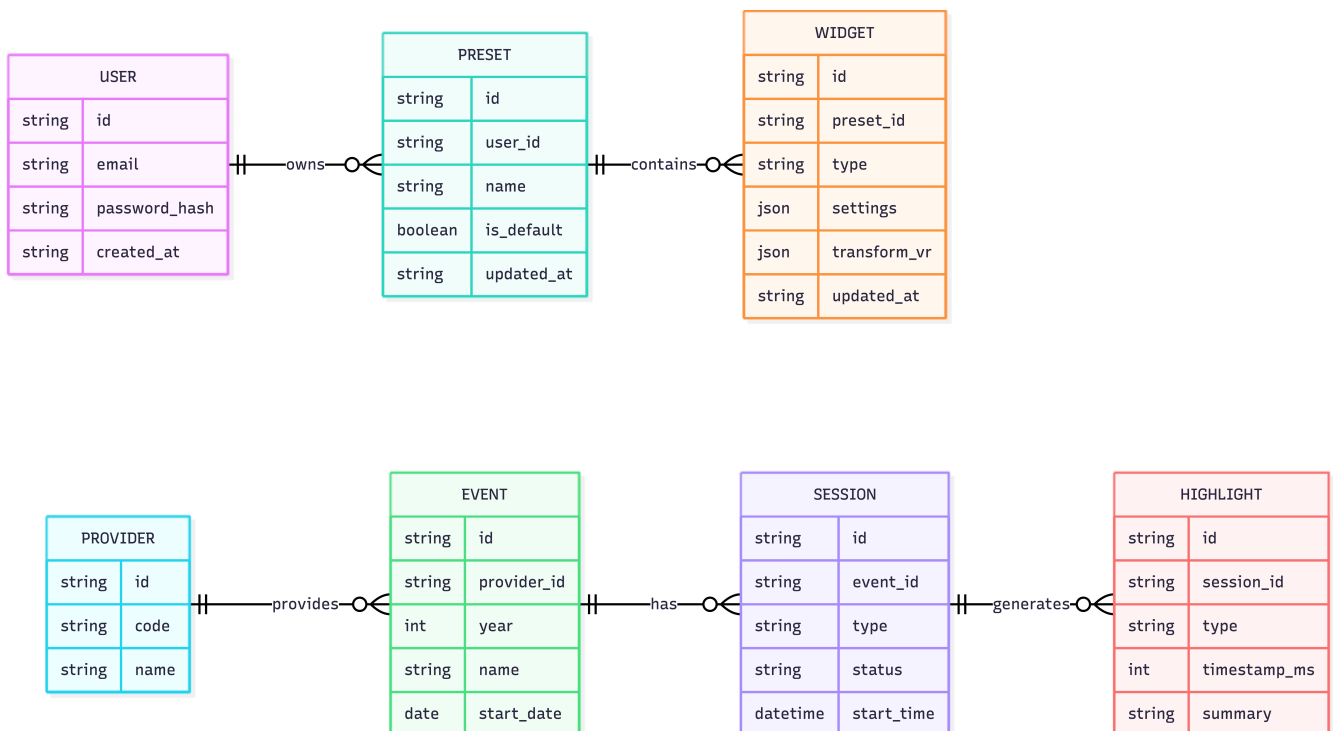
Endpoint principal :

- **GET /sessions/{sessionId}/highlights**

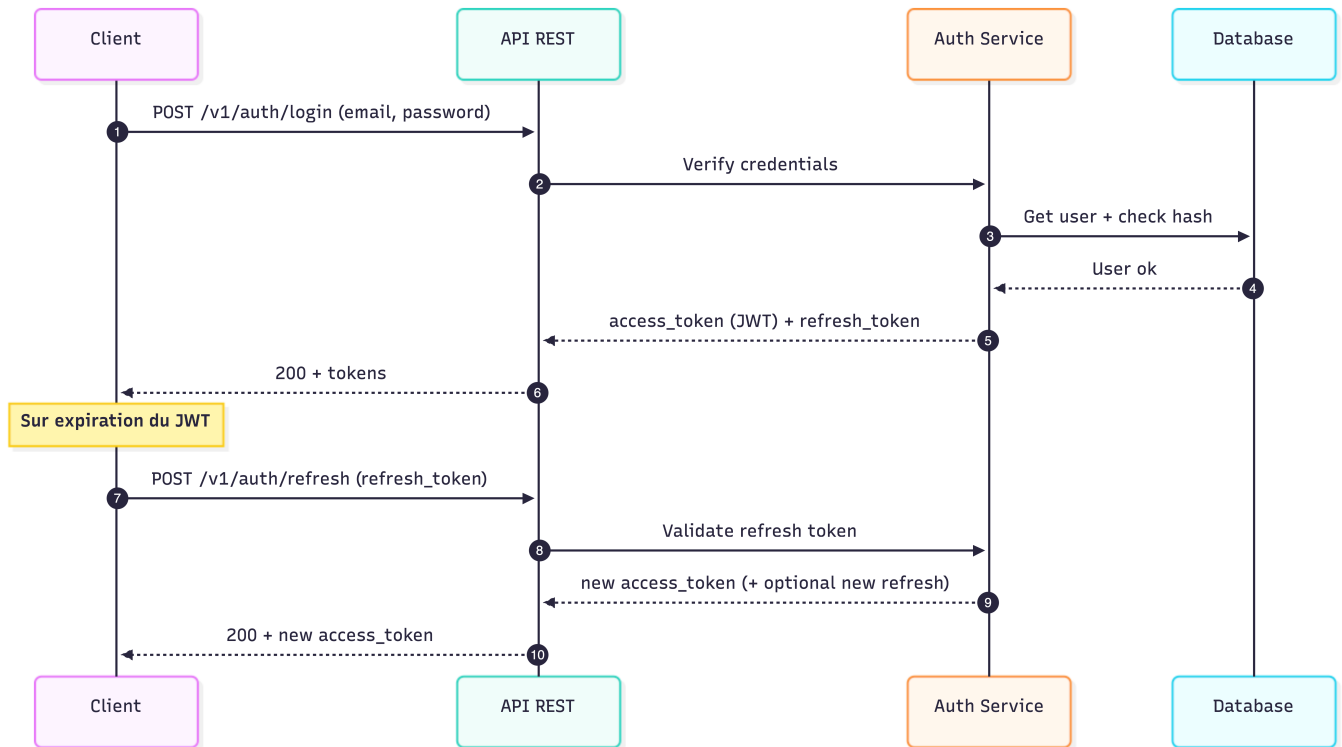
Cette route permet d'afficher :

- une timeline des moments clés,
- un résumé structuré de la session,
- un support pour des replays ou analyses.

6- Schéma : Modèle de données



7- Schéma : Séquence d'authentification



3- WebSocket : diffusion temps réel

Le WebSocket est le canal principal de diffusion live.

Il remplace le polling REST, trop coûteux et trop lent pour des mises à jour fréquentes.

1- Connexion

La connexion WebSocket se fait sur une route dédiée :

- **WS /realtime?sessionId=...**

Lors de la connexion, le client fournit :

- un identifiant de session (sessionId)
- un token d'accès (pour authentifier l'utilisateur)

Ce mécanisme permet :

- de sécuriser l'accès au flux live,
- d'associer correctement le flux à une session précise,
- d'éviter qu'un client écoute des sessions non autorisées.

2- Types de messages envoyés

Le backend envoie des messages structurés, typés, afin d'être facilement interprétables par les clients.

session_state

Ce message représente l'état global de la session.

Exemples d'informations :

- lap actuel,
- drapeau en cours,
- météo,
- track status,
- état global (live, finished).

standings_update

Ce message correspond aux mises à jour du classement.

Il peut contenir :

- positions,
- gaps,
- changements de place,

- informations de stratégie (si disponible).

telemetry_update

Ce message correspond à la télémétrie d'un pilote.

Champs typiques :

- driver_id
- timestamp_ms
- speed
- gear
- throttle
- brake
- (et toute métrique disponible)

highlight_event

Ce message correspond à un événement important détecté ou reçu depuis le provider.

Champs typiques :

- type (overtake, crash, pit, flag)
- timestamp_ms
- summary

3- Gestion de la charge et optimisation

Une contrainte majeure du temps réel est la charge serveur.

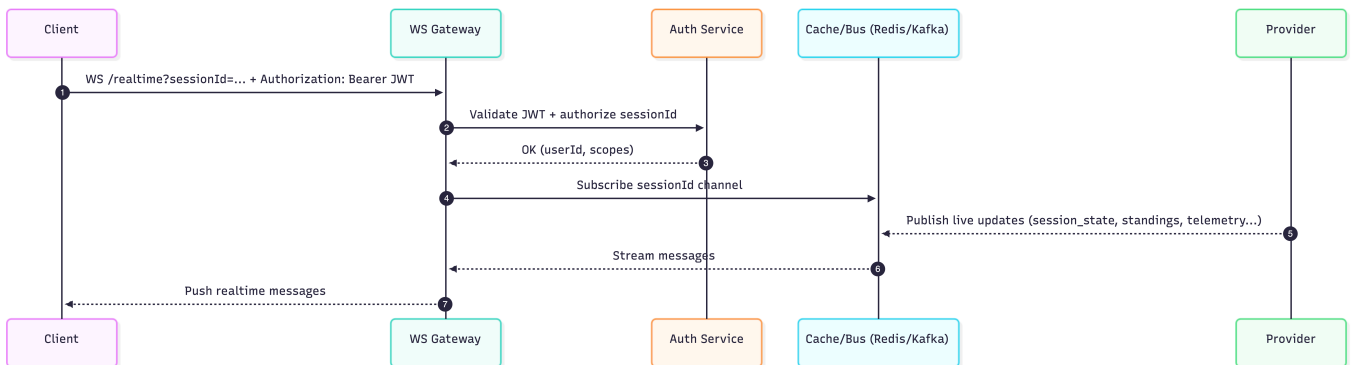
Pour éviter une surcharge, OverDrive peut appliquer une logique de diffusion différenciée :

- fréquence élevée pour le pilote "focus" (ex : suivi du pilote favori),
- fréquence réduite pour les autres pilotes,
- limitation de certains messages selon le contexte (ex : VR vs mobile).

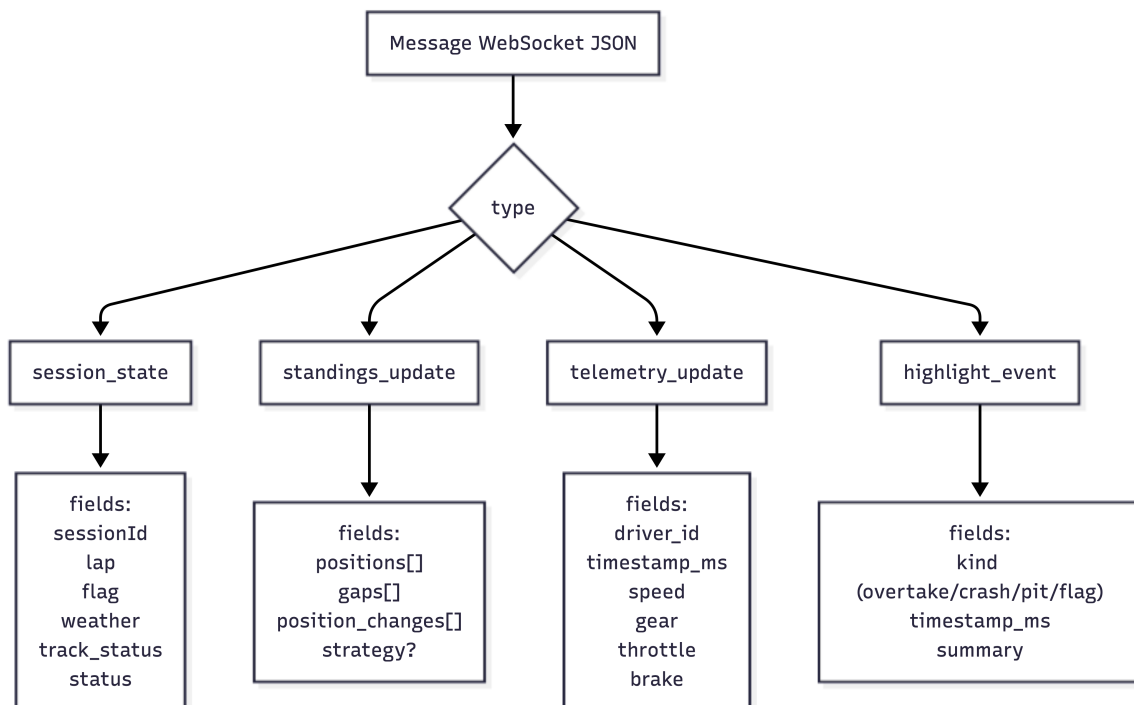
Cela permet de maintenir :

- une expérience fluide,
- une bande passante maîtrisée,
- un coût serveur raisonnable.

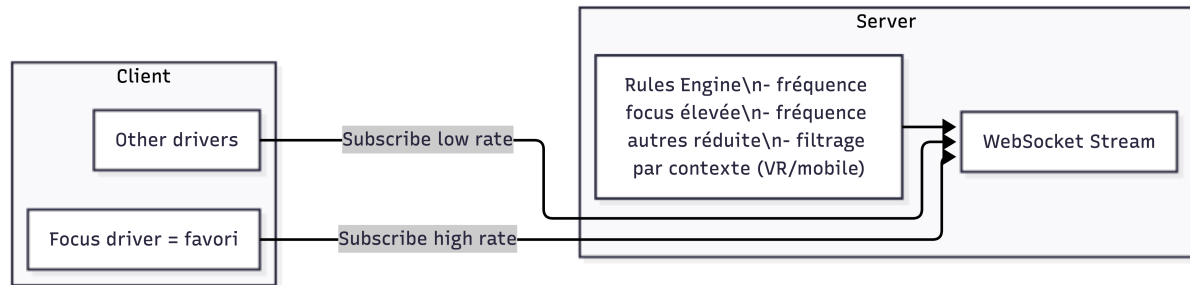
4- Schéma : Connexion WebSocket + souscription à une session



5- Schéma : Format des messages WebSocket



6- Schéma : Optimisation de charge (diffusion différenciée)



4- Principes techniques recommandés

Pour rendre l'API stable, maintenable et scalable, plusieurs bonnes pratiques sont recommandées.

1- Versionnement

Toutes les routes REST doivent être versionnées, par exemple :

- **/v1/auth/...**
- **/v1/events/...**
- **/v1/presets/...**

Le versionnement permet :

- d'évoluer sans casser les clients,
- de gérer plusieurs versions si nécessaire.

2- Documentation (OpenAPI / Swagger)

L'API REST doit être documentée via un standard type OpenAPI, afin de :

- faciliter le développement multi-clients,
- permettre une compréhension rapide des endpoints,
- garantir la cohérence entre backend et clients.

3- Validation et schémas JSON

Le backend doit valider systématiquement :

- les entrées utilisateur (REST),
- les payloads WebSocket.

Cela réduit les risques de :

- bugs,
- incohérences,
- vulnérabilités (injection, crash, payload malformé).

4- Rate limiting (REST)

Même si le WebSocket couvre le live, le REST doit être protégé contre l'abus :

- rate limiting par IP,
- rate limiting par utilisateur,
- limitations spécifiques sur **/auth/login**.

5- Gestion d'erreurs standardisée

Les erreurs doivent suivre un format cohérent, incluant :

- un code HTTP clair,
- un message lisible,
- un identifiant d'erreur si nécessaire,
- une explication structurée.

Cela simplifie le debug et améliore la qualité du produit.

6- Contrôle d'accès

L'API doit garantir :

- qu'un utilisateur ne peut accéder qu'à ses presets,
- que certaines routes sont protégées,
- que la sessionId WebSocket est autorisée,
- que les permissions sont cohérentes (RBAC possible à long terme).

Conclusion

Le schéma d'API proposé pour OverDrive repose sur une architecture claire et standard :

- **REST** pour toutes les données persistantes et la gestion du produit (auth, presets, catalogue, configuration),
- **WebSocket** pour diffuser les données live avec une latence minimale (classement, télémétrie, événements).

Cette approche permet :

- une compatibilité native avec les clients VR, mobile et web,
- une expérience live fluide et premium,
- une architecture évolutive (ajout de providers, nouveaux widgets, nouvelles features),
- une maintenance facilitée grâce aux bonnes pratiques (versionnement, OpenAPI, validation, sécurité).

Ce schéma constitue une base solide pour le développement d'OverDrive et son évolution future.